

Reviewer Pack — Eigenvalue Threshold in SOS Moment Matrices for ACC⁰ Lower B...

Entry 072fac5ab465 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 05:29:28 UTC

Eigenvalue Threshold in SOS Moment Matrices for ACC⁰ Lower Bounds

Entry ID: 072fac5ab465 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 05:29:28 UTC

1. Conjecture

Field A (mathematical branch): SUM_OF_SQUARES_HIERARCHY **Field B** (complexity object): ACC_ZERO_CIRCUIT_COMPLEXITY

Statement:

For any function $f \in \{0,1\}^n$ computable by an ACC⁰ circuit of size s and depth d , the minimum eigenvalue $\lambda_{\min}$ of the SOS moment matrix $M_k(f)$ satisfies $\lambda_{\min} \geq \Omega(s^{\{-1/2\}})$ for $k = \Omega(\log s)$. This implies that any SOS-based approximation of f with error $\varepsilon < \Omega(s^{\{-1/2\}})$ requires $\Omega(\log s)$ levels of the hierarchy, thereby bounding the depth of ACC⁰ circuits computing f .

Rationale (proposer's reasoning):

This conjecture leverages the connection between SOS hierarchies and ACC⁰ circuit complexity, building on Williams' 2011 framework. By analyzing eigenvalue thresholds, it avoids Fourier-based techniques that trigger natural proofs, instead focusing on algebraic geometry and real polynomial approximations. The testable hypothesis ties spectral properties of moment matrices to circuit depth, aligning with the approach's target separation of NEXP $\not\subseteq$ ACC⁰.

Taxonomy category: ACC_LB_via_WILLIAMS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a62ef084d80bb046

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=30.6s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def matrix_multiplication(A, B):
        m, k = len(A), len(B[0])
        result = [[sum(a * b for a, b in zip(row, col)) for col in zip(*B)] for row in A]
        return result

    def gaussian_elimination(matrix):
        n = len(matrix)
        for i in range(n):
            max_row = max(range(i, n), key=lambda r: abs(matrix[r][i]))
            matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
            factor = 1 / matrix[i][i]
            for j in range(i, n):
                matrix[i][j] *= factor
            for k in range(n):
                if k != i:
                    factor = matrix[k][i]
                    for j in range(i, n):
                        matrix[k][j] -= factor * matrix[i][j]
        return matrix

    def compute_eigenvalue(matrix):
        n = len(matrix)
        identity = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
        eigenvalues = []
        A = matrix
        for _ in range(100): # Power iteration method
            v = [random.random() for _ in range(n)]
            v = [x / sum(v) for x in v] # Normalize
```

```

        Av = matrix_multiplication(A, v)
        lambda_ = sum(x * y for x, y in zip(Av, v))
        eigenvalues.append(lambda_)
    return max(eigenvalues)

def sos_moment_matrix(f):
    n = int(math.log2(len(f)))
    M = [[0] * (n + 1) for _ in range(n + 1)]
    for i in range(2**n):
        x = [f[i >> j & 1] for j in range(n)]
        for k in range(n + 1):
            M[k][k] += x[0]**k
            for l in range(k):
                M[l][k] += x[0]**l * x[1]**(k - l)
    return M

n = 20
f = generate_random_function(n)
M = sos_moment_matrix(f)
M = gaussian_elimination(M)
lambda_min = compute_eigenvalue(M)

k = int(math.log2(n))
s = len(f) // (1 << n)
conjecture_holds = lambda_min >= 1 / math.sqrt(s)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "min_eigenvalue",
    "metric_value": lambda_min,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4e3a3179.py", line 92, in <module>
    trial_result = run_trial(seed)
                   ~~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4e3a3179.py", line 72, in run_trial
    lambda_min = compute_eigenvalue(M)
                ~~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4e3a3179.py", line 52, in compute_eigen
    Av = matrix_multiplication(A, v)
        ~~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4e3a3179.py", line 25, in matrix_multip
    m, k = len(A), len(B[0])
           ~~~~~~
```

TypeError: object of type 'float' has no len()

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to type error in matrix multiplication, preventing evaluation of eigen-value threshold. | next: Fix matrix multiplication to handle numeric types, then re-run trials with diverse ACC⁰ circuit parameters

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	51233
2	preregistration	ollama_remote	qwen3:8b	0	0	27508
3	novelty	ollama_remote	qwen3:8b	0	0	24096
4	novelty	ollama_remote	qwen3:8b	0	0	19097
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	43908
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8674
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9112
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11280
9	judge	ollama_remote	qwen3:8b	0	0	53167

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 248076 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/072fac5ab465.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/072fac5ab465.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/072fac5ab465.tar.gz (if generated)